Course Project: From Vanilla RNNs to Attention Mechanisms

Overview

In this project, you will bridge the gap between theory and practice by implementing the core mathematical equations of Sequence Models. You are provided with a working training loop and data generator; your job is to complete the model architectures in Code_Skeleton.ipynb.

Prerequisites: Lectures on RNNs, LSTMs, and Attention (Lecture 13).

Part 1: The Vanilla RNN (The Struggle)

Objective: Implement the fundamental recurrence formula to generate text (Shakespeare).

Reference: Lecture Slide

Your Tasks:

Open Code_Skeleton.ipynb and locate VanillaRNN.

- **TODO 1.1 (__init__):** Define the linear layers.
  - *Concept:* In our implementation, we simplify the math by concatenating input and hidden state first.
  - *Hint:* The input size to your linear layer will be input_size + hidden_size.
- **TODO 1.2 (forward):** Concatenate the input.
  - *Hint:* Use torch.cat((input_tensor, hidden_tensor), dim=1).
- **TODO 1.3 (forward):** Compute the hidden state.
  - Apply your linear layer -> Apply torch.tanh.
- **TODO 1.4 (forward):** Compute the output probabilities.
  - Apply your output layer -> Apply self.softmax.

Analysis:
Run the sanity check. If it passes, the model is valid.

Part 2: The LSTM (The Upgrade)

Objective: utilize PyTorch's optimized nn.LSTM to solve vanishing gradients.

Reference: Lecture Slide (Gated Cells).

Your Tasks:
Locate LSTMGenerator.

- **TODO 2.1 - 2.2 (__init__):** Define the layers.
  - Use nn.LSTM instead of manual math.
- **TODO 2.3 (forward):** Reshape the input.

    - *Critical:* PyTorch LSTMs expect a 3D tensor: (sequence_length, batch, input_features).
    - Since we are processing 1 character at a time with batch size 1, reshape your input using input.view(1, 1, -1).
- **TODO 2.4 (forward):** Pass data through the LSTM.
    - The LSTM returns output and a tuple (hidden_state, cell_state).
- **TODO 2.5 (forward):** Project to output.
    - The LSTM output shape is (1, 1, hidden_size). You need (1, output_size). Access the data using output[0].

Part 3: Neural Translation with Attention (The Breakthrough)

Objective: Implement **Bahdanau Attention** (Additive Attention) to allow the decoder to "look back" at specific parts of the input sentence.

Data: Translating "Human Dates" (July 4, 1776) $\to$ "Machine Dates" (1776-07-04).

Your Tasks:: The Decoder (AttnDecoderRNN)

- **TODO 3.2.1 (__init__):** Define the LSTM.
    - The input to this LSTM is the concatenation of the **Input Word Embedding** and the **Attention Context Vector**.
    - Input Size = hidden_size + hidden_size.
- **TODO 3.2.2 - 3.2.4 (forward):**
    - Calculate attention using self.attention(hidden[0], encoder_outputs).
    - Concatenate the resulting context with the embedded input.
    - Feed the result into the LSTM.

Submission & Report

Code: Submit your completed Code_Skeleton.ipynb. And make sure the code is runnable.

In addition, you are recommended to try the code to see how attention score changes in different sequences. But this is not required in this project assignment.